

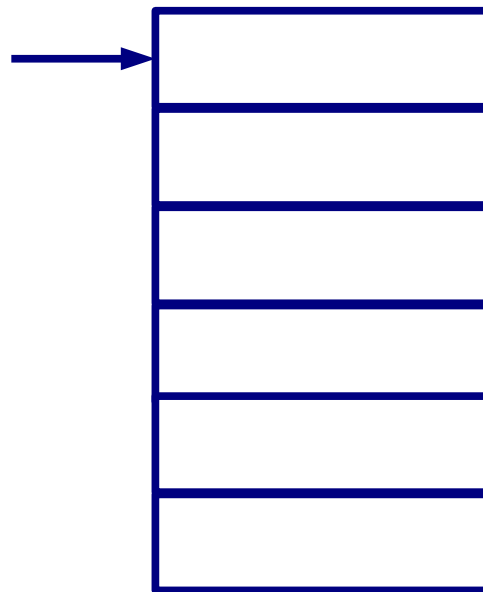
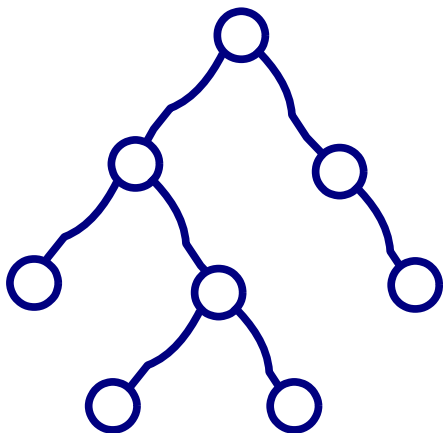
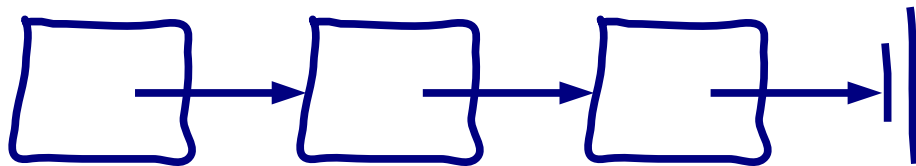
Projeto Detalhado de Software

Padrões de Projetos
(GoF Patterns)

Iterator

Padrão – Iterator

- **Intenção:** Prover acesso a elementos de um objeto agregado de forma sequencial, sem expor a representação interna.

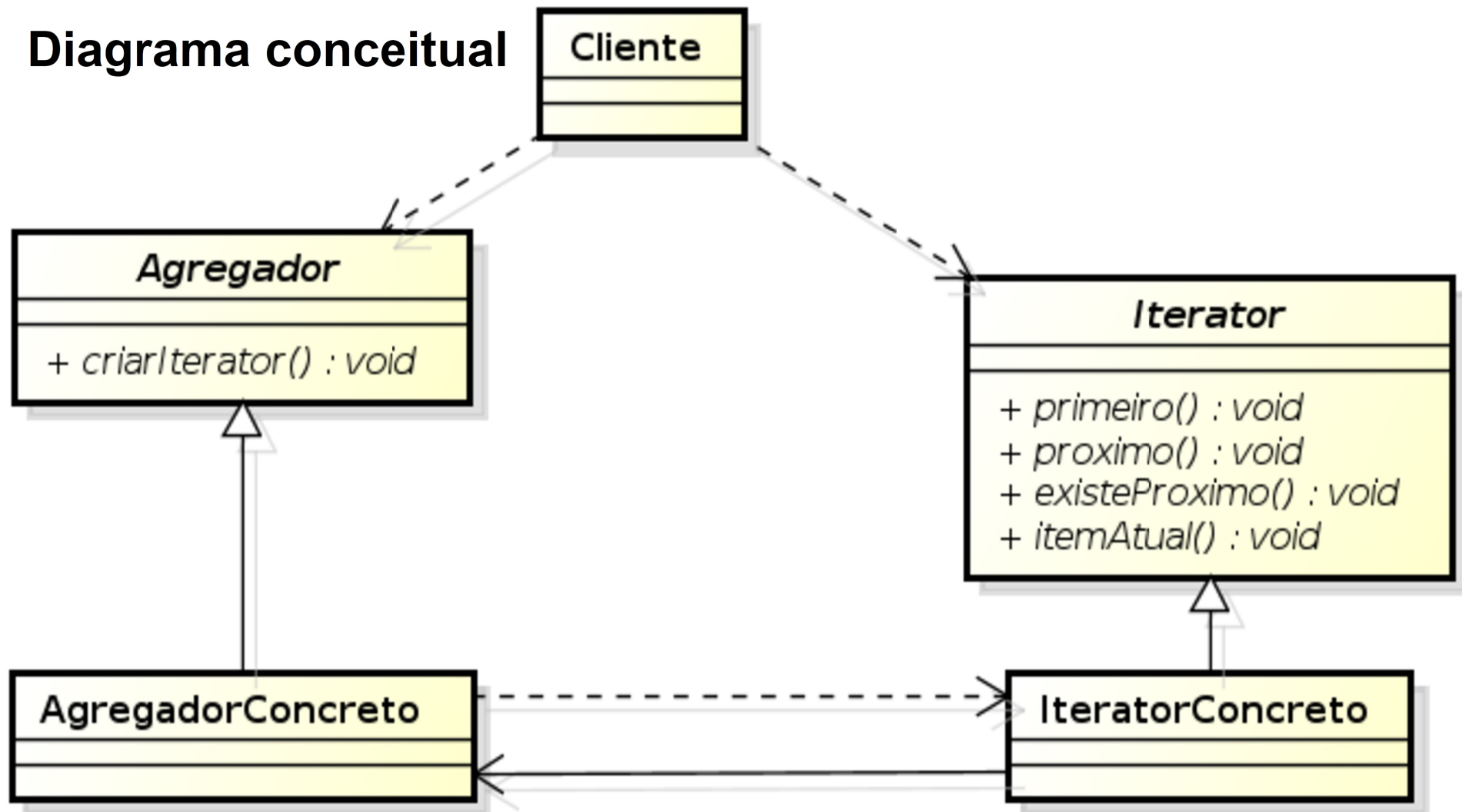


Motivação

Saber a
representação
interna implica
em
ACOPLAMENTO

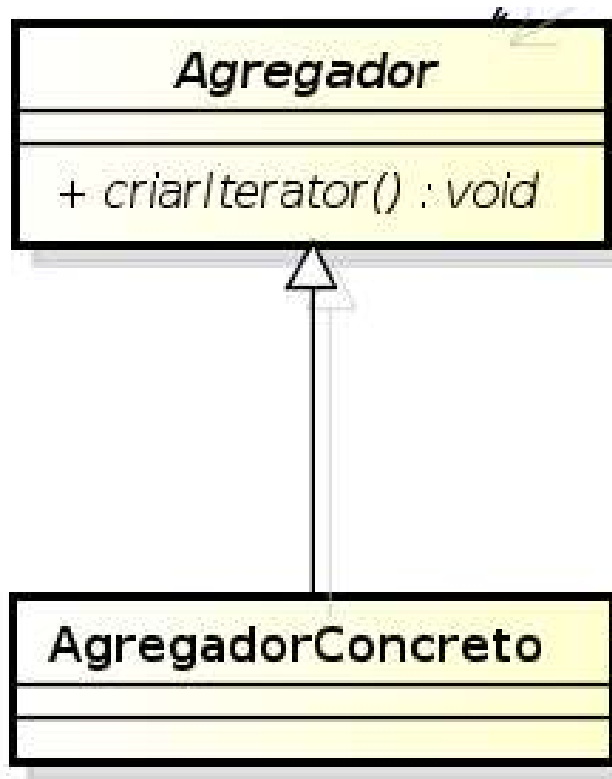
Padrão – Iterator

- Diagrama conceitual



Padrão – Iterator

- Participantes

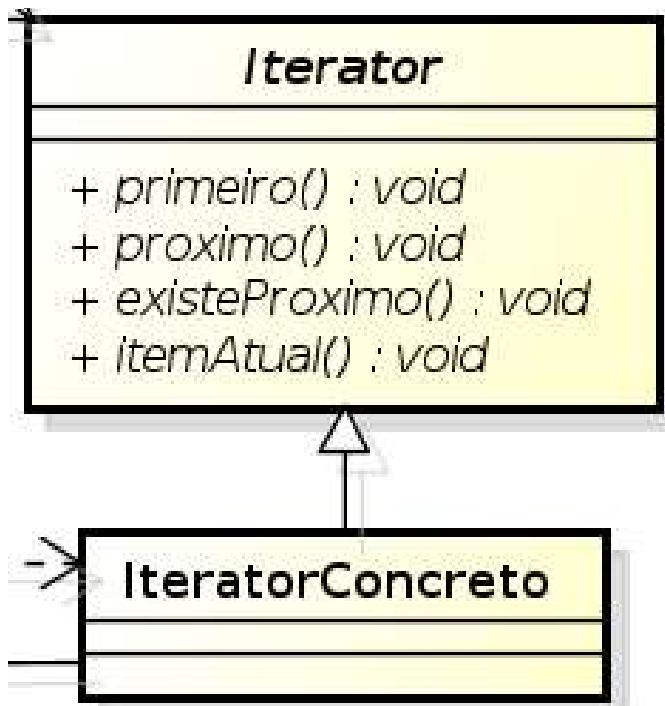


Define uma interface para criação dos objetos Iteradores

Implementa a interface de criação e retorna instância do IteratorConcreto

Padrão – Iterator

- **Participantes**



Define uma interface para acesso e percorrimento dos elementos

Implementa a interface do Iterator, e mantém o estado do percorrimento

Padrão – Iterator

- **Aplicabilidade**

- Encapsular representação interna de uma coleção de objetos
- Desassociar do Cliente as opções de percorrimeto ou filtros
 - Busca em pré-ordem, pós-ordem, em ordem
 - Uso de filtro (predicados)
 - Uso de ordenação

Padrão – Iterator

- **Conseqüência**
 - Suporte a variações no percorrimeto de um agregador
 - Simplificação na interface do Agregador
 - Não é necessário lidar com detalhes das variações
 - Mais de um iterador pode ser anexado a um agregador
 - Cada iterador concreto possui seu estado (cursor)

Padrão – Iterator

- Exemplo (Java Collections Framework)

```
java.lang.Iterable // Agregador (interface)
```

```
– iterator() : Iterator<E>
```

```
java.util.Iterator // Iterator (interface)
```

```
– hasNext() : boolean
```

```
– next() : Object
```

Padrão – Iterator

```
class MenuItem {  
    String nome;  
    MenuItem(String nome) {  
        this.nome = nome;  
    }  
}  
  
interface Iterator {  
    boolean hasNext();  
    Object next();  
}  
  
public class MenuIterator implements Iterator {  
    MenuItem[] itens;  
    int posicao = 0;  
    public MenuIterator(MenuItem[] itens) {  
        this.itens = itens;  
    }  
    public Object next() {  
        MenuItem menuItem = itens[posicao];  
        posicao++;  
        return menuItem;  
    }  
    public boolean hasNext() {  
        if (posicao >= itens.length ||  
            itens[posicao] == null) {  
            return false;  
        } else {  
            return true;  
        }  
    }  
}
```

Padrão – Iterator

```
public class MostraMenu {  
    public static void main(String args []) {  
        MenuItem [] menuItems = new MenuItem[4];  
  
        menuItems[0] = new MenuItem("Menu 1");  
        menuItems[1] = new MenuItem("Menu 2");  
        menuItems[2] = new MenuItem("Menu 3");  
        menuItems[3] = new MenuItem("Menu 4");  
  
        for (int i=0; i < menuItems.length; i++) {  
            System.out.println(menuItems[i].nome);  
        }  
    }  
}
```

Padrão – Iterator

- Exercício (?? min)
 - ArrayList<E> possui uma implementação de Iterator
 - Retorna os elementos na ordem em que foram inseridos
 - A fazer = criar uma MinhaArrayList<E> que possua 3 novos tipos de iteradores:
 - iteratorReverso()
 - IteratorValoresPares()
 - IteratorValoresImpares()

Conclusão

- Iterator

**ENCAPSULA REPRESENTAÇÃO INTERNA DE
COLEÇÕES**

**PROMOVE A AGREGAÇÃO DE FILTROS E OPÇÕES DE
PERCORRIMENTOS**

Referências

- **“Padrões de Projeto: Soluções Reutilizáveis...”** Erich Gamma et al. Padrão Iterator
- <https://www.devmedia.com.br/padrao-de-projeto-iterator-em-java/26733>